

講義 25：Data Dictionary 總覽與 Global Type (授課順序：接在講義 7 之後、講義 21 之前)

對應練習：ex25 | 答案物件：Domain/DE ZTR25_SURPCT、DE ZTR25_ACTIVE (重用標準 Domain XFELD)、表 ZTR25_SURCHG (SM30)、Table Type ZTR25_TT_SURCHG + 程式 ZR_TR25_DDIC

本講重點

- Data Dictionary (SE11) 在系統裡的角色：一張物件地圖，程式與畫面共用同一份定義
- **DDIC 內建型別** (DEC、CHAR、CURR.....) 跟講義 2 的 ABAP 內建型別有何不同、怎麼對應
- **Global Type** 回顧 (觀念已在講義 6 第 1.1 節講過)：重點放在動手建自己的 Global Type
- 為什麼要自建 Z 表：業務需求 + SM30 讓非工程師也能維護資料
- Check Table (檢查表) / Foreign Key (外鍵) / Search Help (搜尋輔助)：概念總覽 (詳細動手在講義 21)
- 程式裡引用表格 / 結構的三種寫法，以及「該重用標準型別、還是自建」的判斷
- **DDIC Table Type**：把講義 4 的區域表格型別升級成全域可共用的 SE11 物件
- 完整案例：SM30 維護的「航空公司旺季加成」設定表，重用標準 Data Element 免費拿到 Check Table 與 Search Help

1. Data Dictionary 是什麼：一張系統地圖

從講義 6 起你一直在用 DDIC (TYPE scarr-carrid)，但還沒看過它的全貌。SE11 管理的物件其實有一整組層級，各司其職：

物件	管什麼	誰在用
Domain (值域)	技術屬性：型別、長度、小數位、值域清單	Data Element
Data Element (資料元素)	語意：標籤、F1 說明、Search Help 掛勾 (講義 21 三層件)	表格欄位、程式變數
Structure (結構)	純欄位組合， 不對應資料庫表 (如畫面用的暫存結構)	程式、畫面
Table (透明表)	對應資料庫的真實表	Open SQL、SM30、程式
Table Type	表格型別 (「很多列」的定義)，可跨程式共用	方法/FM 的表格參數
View (檢視)	把一張或多張表的欄位「組合」起來看，本身不存資料 (四種，見 1.1)	程式、SM30、Search Help
Search Help	F4 選單來源	畫面欄位、Data Element
Lock Object	產生 ENQUEUE/DEQUEUE FM，防止多人同時改同一筆 (講義 21 提過，進階課題)	程式

關鍵觀念：這些定義只寫一次，程式（用 **TYPE**）跟畫面（**Dynpro/SM30**）共用同一份——這就是接下來 Global Type 觀念的基礎。

用航空公司代碼 **CARRID** 當例子。它在 DDIC 裡只定義一次：

- Domain **S_CARR_ID**：技術屬性，**CHAR** 長度 3
- Data Element **S_CARR_ID**：語意，包含欄位標籤、F1 說明，以及掛好的 Search Help **S_CARRIER_ID**（F4 選單）

這裡 Domain 和 Data Element 剛好同名，都叫 **S_CARR_ID**，但它們是兩個不同的物件。程式裡 **TYPE s_carr_id** 引用的是 **Data Element**，不是 Domain：Domain 只給 Data Element 用，程式不能直接 **TYPE** 一個 Domain。

程式宣告型別常見三種寫法：前兩種宣告單一欄位，結果相同；第三種宣告整列結構。

寫法	引用的是	說明
TYPE s_carr_id	Data Element S_CARR_ID	直接指定 Data Element 名稱
TYPE scarr-carrid	表格欄位 SCARR-CARRID	取這個欄位的型別；而 SCARR-CARRID 本身就是用 Data Element S_CARR_ID 定義的，所以最後拿到的一樣
TYPE scarr	表格 SCARR 的結構（一列）	例如 DATA gs_scarr TYPE scarr. ，宣告一個結構變數，欄位跟 SCARR 完全一樣（ carrid 、 carrname 、 currcode 、 url），每個欄位的型別也都來自各自的 Data Element；存放 SELECT SINGLE * FROM scarr 讀出的一整列時用這種

課程裡兩種都會看到：跟某張表的欄位對應時（例如要存放 **SCARR-CARRID** 讀出的值），用 表格-欄位 比較直觀；沒有特定對應的表，只是要一個「航空公司代碼」型別時，用 Data Element。

之後所有地方都引用這一份：

誰在用	怎麼引用	拿到什麼
程式變數	DATA gv_carrid TYPE s_carr_id. （Data Element）或 TYPE scarr-carrid （表格欄位）	型別、長度自動是 CHAR 3
選擇畫面	PARAMETERS p_carr TYPE s_carr_id. （Data Element）或 TYPE scarr-carrid （表格欄位）	型別、長度，加上 F1 說明、F4 選單，不用自己寫
表格欄位	SCARR-CARRID 、 SPFLI-CARRID 、 SFLIGHT-CARRID 在 SE11 都指定 Data Element S_CARR_ID	同一個型別，JOIN 時兩邊保證對得上
SM30 維護畫面	系統依表格欄位自動產生	欄位標題、F1、F4 都從欄位的 Data Element 帶出來

「畫面」這一邊（選擇畫面、Dynpro、SM30）的欄位標題、F1、F4，都不是畫面自己定義的，而是從 DDIC 帶過來。

好處在修改時最明顯：如果長度要從 3 改成 4，只要改 Domain 一個地方，所有引用它的程式、畫面、表格都跟著變。反過來，如果每支程式都自己寫 **TYPE c LENGTH 3**，就要一支一支找出來改，漏改一支就會出錯（值

被截斷、型別對不上) 。

這種「在 SE11 定義、全系統都能引用」的型別就叫 **Global Type**；相對的是講義 3、4 教的 Local Type (程式裡的 **TYPES**)，只在那支程式裡有效。本講接下來就是教怎麼建自己的 Global Type。

1.1 View (把表的欄位組合起來看，本身不存資料) 的四種類型

View 不存資料，只是把表的欄位組合起來。SE11 → **View** 建立時要先選類型，四種用途完全不同：

類型	做什麼	表怎麼連	程式能 SELECT 嗎	系統標準範例	詳見
Database View	多張表 JOIN 成一個物件，程式直接讀	INNER JOIN，條件自己定 (可參考外鍵帶出)	✔ 可以	SFLIGHTS (SCARR + SPFLI + SFLIGHT)	講義 11 §2.1
Projection View	只露出一張表的部分欄位	只有一張表	✔ 可以	DEMO_SPFLI (SPFLI 部分欄位)	本節
Maintenance View	讓 SM30 一次維護多張相關的表 (例如主檔 + 文字表)	INNER JOIN，必須沿用外鍵	✗ 不行	V_TCURC (幣別 TCURC + 文字 TCURT)	講義 21a §3
Help View	當 Search Help 的資料來源	OUTER JOIN，必須沿用外鍵	✗ 不行	H_T005 (國家 T005 + 文字 T005T)	講義 21a §4

- 四種都能當 **TYPE** 用 (`DATA gs_x TYPE v_tcurc.`)，因為 View 在 DDIC 裡也定義了一個結構。
- 對 Maintenance View 或 Help View 寫 **SELECT**，啟用時直接報錯 (2026-09-29 實測)：`"V_TCURC" is not declared as a table, projection view, or database view in ABAP Dictionary`。錯誤訊息本身就列出了能 **SELECT** 的只有「表、Projection View、Database View」三種。
- 只有 Database View 會在資料庫真的建一個 SQL View；另外三種只存在 DDIC，由 SAP 自己處理。
- S/4HANA 之後，要在程式裡讀多表組合，SAP 建議改用 **CDS View** (CDS 課程)，傳統 Database View 以維護舊程式為主。Maintenance View 與 Help View 則沒有被取代，SM30 和 Search Help 仍然常用。
- 在這個系統，傳統 View 只能用 **SE11** 建立和查看，ADT / Eclipse 讀不到 (2026-09-29 實測，讀 `SFLIGHTS` 回傳錯誤)，跟 Search Help 一樣屬於 GUI 操作。

示範程式 `ZR_TR25_VIEW_DEMO` (`$TMP`，快照 `zr_tr25_view_demo.prog.abap`)：讀 `SFLIGHTS`、`DEMO_SPFLI`，並把 `V_TCURC`、`H_T005` 當型別使用。

Projection View 跟「直接從表 **SELECT** 幾個欄位」有什麼差別？

先講結論：讀資料的效能完全一樣。Projection View 在資料庫上不會建立任何 View，程式 **SELECT** 它的時候，系統把它轉成「從原表讀那幾個欄位」的 SQL (官方文件 `ABENDDIC_PROJECTION_VIEWS`)。下面兩句對資料庫來說是同一件事：

```
SELECT * FROM demo_spfli INTO TABLE gt_a. " 透過 Projection View
SELECT carrid connid cityfrom cityto FROM spfli INTO TABLE gt_b. " 直接列出同樣的欄位
```

DEMO_SPFLI 只露出 SPFLI 的 CARRID、CONNID、CITYFROM、CITYTO (加上 client 欄位 MANDT)，出發時間、飛行距離等其他欄位都看不到；它的 Maint. Status 是 Read only (2026-09-30 查 DD25L / DD27S)。

差別不在效能，而在「欄位清單定義在哪裡」：

	直接從表 SELECT 欄位	透過 Projection View
欄位清單寫在哪	每支程式各寫一次	在 SE11 定義一次，多支程式共用
程式要一個「只有這幾欄」的結構	自己寫 TYPES BEGIN OF ... END OF	直接 TYPE demo_spfli
表加了新欄位	不受影響	不受影響 (View 只露出定義的欄位)
寫入	對整張表 INSERT / UPDATE	Maint. Status 設成可寫時，也能透過 View 寫；沒列在 View 裡的欄位會填初始值
限制只能讀	做不到	Maint. Status 設成 Read only ，透過這個 View 就只能讀

所以 Projection View 的用途是：把「一張表對外只開放哪幾個欄位」定義成一個 **DDIC** 物件，讓多支程式共用同一份欄位清單與結構型別，也可以做成唯讀的存取入口。

但要注意兩點：

- 它不是權限控管。程式照樣可以直接 **SELECT** 原表的所有欄位，Projection View 擋不住。要控管誰能看什麼，要用權限檢查 (講義 28)。
- 實務上自建的很少。要「只讀幾個欄位」，直接在 **SELECT** 列出欄位就好；要共用結構，建一個 DDIC Structure 也可以。官方文件也說明，傳統 View 能做的事 CDS View 都做得到而且更多 (CDS 課程)。學 Projection View 主要是為了看懂標準系統與舊程式裡既有的 View。

1.2 DDIC 內建型別 vs ABAP 內建型別

講義 2 學的 **i**、**p**、**c**、**d** 是 **ABAP** 內建型別，寫在程式裡。SE11 建 Domain 時選的 **DEC**、**CHAR**、**DATS** 是 **DDIC** 內建型別，兩者是不同的東西：

	ABAP 內建型別 (講義 2)	DDIC 內建型別
用在哪裡	程式： DATA gv TYPE p ...	SE11：Domain、表格欄位
寫法	小寫： i 、 p 、 c 、 d	大寫： INT4 、 DEC 、 CHAR 、 DATS
程式能不能直接寫	可以	不行，只能透過 Data Element、表格欄位等 DDIC 物件間接引用 (官方文件稱為「外部型別」)
另一個角色	-	對應到資料庫欄位的型別；DDIC 負責在 ABAP 型別與各家資料庫型別之間轉換

程式寫 `DATA gv_pct TYPE ztr25_surpct.` 時，系統把 Data Element 背後 Domain 的 DDIC 型別，轉成對應的 ABAP 型別。常用的對應 (官方文件 `ABENDDIC_BUILTIN_TYPES`)：

DDIC 型別	對應的 ABAP 型別	說明
CHAR m	c LENGTH m	字元
NUMC m	n LENGTH m	數字字元 (編號)
STRING / SSTRING	string	變動長度字串
INT4	i	4 bytes 整數
INT1 / INT2 / INT8	b / s / int8	1、2、8 bytes 整數
DEC m,n	p LENGTH m DIV 2 + 1 DECIMALS n	壓縮十進位
FLTP	f	浮點數
DATS	d	日期 YYYYMMDD
TIMS	t	時間 HHMMSS
RAW / RAWSTRING	x / xstring	位元組

⚠ **DEC** 的長度是「數字位數」，**ABAP p** 的 **LENGTH** 是「bytes」，而且都不含小數點。以本講案例為例 (2026-09-30 用測試程式實測)：

Domain	程式裡的型別	能存的最大值
DEC 5 位、2 位小數	p LENGTH 3 DECIMALS 2	999.99 (1000.00 會溢位 CX_SY_CONVERSION_OVERFLOW)
DEC 6 位、2 位小數 (ZTR25_SURPCT)	p LENGTH 4 DECIMALS 2	9999.99

DDIC 還多了一些帶語意的型別。技術上一樣對應到 `c`、`n`、`p`，但系統會依語意做額外的事：

DDIC 型別	對應 ABAP 型別	多出來的意義
CURR	p	金額，必須指定一個 CUKY 型別的參考欄位；顯示時依該欄位的幣別決定小數位 (例如日圓沒有小數，見講義 21 §2.2)
CUKY	c LENGTH 5	幣別代碼 (TWD、USD)，給 CURR 參考
QUAN	p	數量，必須指定一個 UNIT 型別的參考欄位；顯示時依該欄位的單位決定小數位
UNIT	c LENGTH 2 ~ 3	單位 (PC、KG)，給 QUAN 參考
CLNT	c LENGTH 3	Client；表格第一欄用它，Open SQL 就會自動處理 client (講義 6)
LANG	c LENGTH 1	語言代碼；文字表靠它依登入語言取說明

DDIC 型別	對應 ABAP 型別	多出來的意義
ACCP	n LENGTH 6	會計期間 YYYYMM

講義 2 的 **p** 只是一個數字，不知道自己是金額；DDIC 的 **CURR** 知道自己是金額，所以一定要有幣別欄位配對。講義 15 建結構 **ZTR15_FLIGHT_REV** 時，金額欄位沒有指定參考幣別就無法啟用，就是這個規則。

2. Global Type (在 DDIC 定義、全系統都能引用的型別) 回顧：為什麼要引用 DDIC 型別，不要寫死

Global Type 的觀念 (內建型別 / Local Type / Global Type 三層、判斷準則、為什麼不寫死長度) 已經在講義 6 第 1.1 節講過，這裡不重複，只回顧最關鍵的一點，接著把重心放在「怎麼動手建自己的 Global Type」。

2.1 回顧：寫死長度是最陰險的 bug

物料號碼 **MATNR** 從 18 碼加長到 40 碼：引用 Global Type 的程式 (**TYPE mara-matnr**) 升級後一行都不用改；寫死 **TYPE c LENGTH 18** 的程式資料被默默截斷，不當機、不報錯，可能幾個月後才被發現。型別的定義權交給系統唯一的來源 (**Domain / Data Element**)，程式只負責「引用」——本講後面的每一種做法，都是在這個前提下，決定「引用現成的」還是「自己建一個」。

2.2 什麼時候該重用標準型別、什麼時候該自建

情境	做法
語意跟標準欄位完全一樣 (航空公司代碼、物料號碼、客戶編號...)	重用標準 Data Element (TYPE scarr-carrid 或直接 TYPE s_carr_id)
你們公司獨有的業務概念，標準系統沒有對應語意 (自訂的加成比例、自訂的審核狀態...)	自建 Domain/Data Element (講義 21 的三層件流程)

判斷原則很簡單：先用 **T-code SE84** 搜尋標準有沒有語意相符的 **Data Element**，有就重用；沒有才自建——本講最後的案例會示範重用標準型別能省下多少工。

SE84 (Repository Information System) 的搜尋方式：

1. 左邊樹狀選單展開 **ABAP Dictionary** → 雙擊 **Data Elements**。
2. 在說明文字 (Short Description) 欄位輸入關鍵字，前後加 *，例如 ***Airline***；也可以在 Data Element 名稱欄位輸入 **S_CARR*** 這種樣式。
3. 執行 (F8)，從清單挑語意相符的，雙擊進去確認型別、長度與欄位標籤。

另一個常用的找法：想存的欄位如果標準表裡已經有 (例如物料號碼在 **MARA**、航空公司代碼在 **SCARR**)，直接用 **SE11** 打開那張表，看該欄位的 **Data Element** 欄就知道要用哪一個。

SE84 畫面欄位名稱依一般 SAP GUI 版本整理，如果你看到的畫面不一樣，請回報，講義會再修正。

3. 為什麼要自建 Z 表：業務需求 + SM30 開放維護

標準系統管不到的東西，例如客製化的參數設定、控制開關、公司自己的業務對照表，就需要自建 Z 表。核心價值不是「能存資料」(那用什麼工具都能存)，而是：

- **SM30** 讓不會寫程式的人也能維護資料：業務人員自己在 SM30 改參數、加一筆設定，不需要工程師介入、不需要走傳輸請求（資料本身不需要 TR，只有表結構變更才需要）。
- 跟講義 21 的分工：講義 **21** 教你「怎麼建」（Domain/DE/Table/外鍵/Search Help 手把手流程，SM30 維護畫面在講義 21a）；本講先講「為什麼建、什麼時候該重用 vs 自建」，兩講合起來才是完整的 DDIC 觀念。

4. Check Table（檢查表） / Foreign Key（外鍵） / Search Help（搜尋輔助）：總覽先看懂

講義 21 會帶你完整動手建一次（兩張都是自建的 Z 表）；這裡先看懂三個名詞在解決什麼問題，本講最後的案例則刻意示範另一種更常見的組合：

名詞	解決什麼問題
Check Table（檢查表）	誰是「合法值清單」——可以是自建的 Z 表，也可以直接是標準表（如 SCARR）
Foreign Key（外鍵）	讓畫面輸入時擋掉不在合法清單裡的值（只管畫面，不管 Open SQL——講義 21 用程式驗證 Open SQL 不被擋，講義 21a 在 SM30 驗證畫面會擋）
Search Help（搜尋輔助）	給 F4 選單；如果欄位重用了已經掛好 Search Help 的標準 Data Element ，你完全不用自己建

SE11 畫面上還會看到這些相關名詞，先認得中英文，之後操作時才對得起來：

英文（SE11 畫面上的用詞）	中文	意思
Foreign Key Table	外鍵表	帶外鍵欄位的那張表（例如學生表）
Check Table	檢查表	被參考、提供合法值的那張表（例如班級表）
Cardinality	基數	兩表資料是幾對幾，例如 1 : CN = 每個學生一個班級、每個班級任意多個學生（意義與填法見講義 21 §3.1）
Foreign key field type	外鍵欄位類型	外鍵欄位在外鍵表裡是不是主鍵；選 text table 時外鍵表會被當成檢查表的文字表（講義 21 §3.1）
Screen Check	畫面檢查	外鍵的開關；打開後畫面輸入才會被檢查
Value Table	值表	設在 Domain 上的建議檢查表；建外鍵時系統會先帶出來當預設
F4 Help / Value Help	F4 說明 / 值說明	在欄位上按 F4 跳出的選單
Selection Method	選取方法	Search Help 從哪張表（或 View）讀選單資料
Search Help Parameter	搜尋輔助參數	Search Help 選單裡的欄位；Import 把畫面值帶進去當條件、Export 把選到的值帶回畫面

本講案例：Check Table 直接指向標準表 **SCARR**，欄位重用標準 Data Element **S_CARR_ID**——順便驗證它已經掛好的 Search Help 直接生效，一個都不用自己建，比講義 21（兩個都要自建）省事得多，也更貼近實務常態

(大部分自建表的關聯欄位，另一端往往是標準主檔) 。

5. 程式中引用 DDIC 物件的三種寫法

```
DATA gs_row TYPE ztable.           " 整列：跟表格結構同型別 ( work area )
DATA gv_val TYPE ztable-field.     " 單一欄位：跟表格該欄位同型別
DATA gt_rows TYPE STANDARD TABLE OF ztable.  " 整張表：多筆集合 ( 講義 4 學過的表格型別 )
```

欄位型別還有一種更直接的寫法——**繞過表格路徑，直接引用 Data Element**：

```
DATA gv_carrid TYPE s_carr_id.      " 直接引用 Data Element
DATA gv_carrid TYPE scarr-carrid.   " 透過表格路徑引用 ( 兩者型別完全相同，因為 SCARR-
CARRID 本來就是用 S_CARR_ID 定義的 )
```

兩種寫法效果一樣，選哪個看語境：如果變數的意義跟某張特定表綁得很緊，用 **TYPE 表-欄位** 讀起來更清楚；如果是通用的「一個航空公司代碼」，直接 **TYPE s_carr_id** 更能表達「這是一個標準概念，不是某張表專屬的」。

6. 案例：SM30 維護的「航空公司旺季加成」設定表

情境：財務單位想針對特定航空公司的航班設定「旺季加成百分比」，希望自己在 SM30 維護、不用每次找工程師改程式；報表要能讀這張表，把加成反映進營收試算。

6.1 建立 Domain 與 Data Element (自建——因為「加成百分比」是我們公司獨有的概念)

- **Domain ZTR25_SURPCT**：Data Type **DEC**，Length **6**、Decimals **2** (Length 是數字位數、不含小數點：6 位含 2 位小數，最大 **9999.99**，見 1.2 節)；Value Range 頁籤設 Interval **0 ~ 100** (DEC 型別的值域上下限只接受整數，填 **0.00 / 100.00** 會啟用失敗) → 啟用
- **Data Element ZTR25_SURPCT**：參考上面的 Domain；Field Label 填「旺季加成百分比」→ 啟用

6.2 第三種 Global Type 模式：重用標準 Domain、自己補標籤

ACTIVE (是否啟用加成) 圖方便的話很容易直接用內建型別 **CHAR 1** 打發——**這是個陷阱**：SE11 建表時該欄位沒有任何 Data Element 可以提供標籤，SM30 產生的維護畫面欄位標題會顯示成通用符號 **+**，而且完全沒有 F4 下拉選單 (沒有 Domain 固定值清單可以參考)。

正確做法：SAP 標準已經有一個通用的「是 / 否」值域 **Domain XFELD**，內建 **X = 是**、空白 = 否兩個固定值——但它掛的標準 Data Element **XFELD** 本身故意不帶任何標籤 (設計上就是留給每張表自己命名，因為「是 / 否」用在哪張表都語意不同)。所以我們建一個新 Data Element **ZTR25_ACTIVE**，**Domain** 選標準的 **XFELD** (不自建 **Domain**)，只補上自己的標籤「加成啟用」：

層	做法
Domain	重用標準 XFELD (技術屬性 + 固定值清單，一個都不用自己建)
Data Element	自建 ZTR25_ACTIVE (只補標籤，型別/值域全部繼承自 XFELD)

這是跟前面兩種模式都不同的**第三種 Global Type 用法**：

模式	例子	重用什麼	自建什麼
整個 Data Element 重用	CARRID 用 S_CARR_ID	型別 + 標籤 + Search Help 全部	什麼都不用建
完全自建	SURCHARGE_PCT 用 ZTR25_SURPCT	什麼都不重用	Domain + Data Element 都自己定義
重用 Domain、自建標籤	ACTIVE 用 ZTR25_ACTIVE (Domain 是 XFELD)	技術屬性 + 固定值清單	只補一個貼近自己業務語境的標籤

判斷原則：遇到「是 / 否」「啟用 / 停用」這類通用的旗標概念，先搜尋標準有沒有像 XFELD 這樣的通用 Domain——有的話重用它的技術屬性與固定值，自己只需要補標籤，比整組自建省事得多。

6.3 建立表 ZTR25_SURCHG

欄位	Key	型別來源	說明
MANDT	✓	Data Element MANDT	client
CARRID	✓	標準 Data Element S_CARR_ID (不是自建！)	航空公司代碼，同時是 Key 也是外鍵
ACTIVE		Data Element ZTR25_ACTIVE (6.2 自建，重用標準 Domain XFELD)	是否啟用加成
SURCHARGE_PCT		Data Element ZTR25_SURPCT	加成百分比 (自建)
UPDUSER		Data Element SYUNAME	異動者
UPDDATE		Data Element SYDATUM	異動日

Key 與 Initial Values 兩個勾選框：Key 勾起來的欄位合起來是這張表的主鍵，用來唯一識別一筆資料；這張表一家航空公司只有一筆設定，所以主鍵是 MANDT + CARRID，而且 Key 欄位必須連續排在最前面。Key 右邊還有一格 Initial Values (欄寬窄時顯示 Init...，DDL 寫成 not null)，表示資料庫欄位不允許 NULL，沒給值時填初始值 (空白、0)。勾了某個欄位的 Key 之後，系統會自動把同一列的 Initial Values 也勾起來 (Key 欄位一定不能是 NULL)；這張表不是 Key 的欄位 (ACTIVE、SURCHARGE_PCT、UPDUSER、UPDDATE)，Initial Values 維持不勾即可。什麼是 NULL、什麼情況一定要勾 (在已有資料的表加新欄位)，主鍵的完整規則，下一講講義 21 §2.1 詳細說明。

Currency/Quantity Fields 頁籤不用填：表裡如果有金額 (CURR) 或數量 (QUAN) 欄位，要在這個頁籤填 Reference table / Ref. field，指定它的幣別或單位欄位，否則表無法啟用 (講義 21 §2.2)。這張表的加成百分比 SURCHARGE_PCT 是 DEC 型別：百分比不是金額，沒有幣別，所以用 DEC 而不用 CURR，這個頁籤就不用填。

CARRID 欄位的 Foreign Key 對話框：Check Table 填 SCARR (標準表，不是自建的 Z 表！)，基數 (Cardinality) 填 1 : C (每筆加成設定對應一家航空公司；每家航空公司最多一筆加成設定，也可以沒設定，所以是 C。說明見講義 21 §3.1)，Foreign Key Fields 讓系統自動帶出 CARRID = CARRID；打開 Screen Check。存檔啟用。

DDL 檢視長這樣 (跟講義 21 的外鍵語法同一套，只是 Check Table 換成標準表)：

```
key carrid : s_carr_id not null
  with foreign key [0..1,1] scarr
    where mandt = ztr25_surchg.mandt
      and carrid = ztr25_surchg.carrid;
```

6.4 建 SM30 維護畫面、驗證「免費拿到」的東西

- 先建 **Function Group** : SE80 → 下拉選 **Function Group** → 輸入 **ZFG_TR25** → Enter → Yes → 填 Short Text → 套件 **\$TMP**。Table Maintenance Generator 產生的維護畫面程式要放進一個已經存在的 Function Group，所以要先建好（Function Group 是 FM 的容器，講義 15 會細講）
- Utilities → Table Maintenance Generator : Authorization Group **&NC&**、Function Group **ZFG_TR25**、one step → 產生
- SM30 → **ZTR25_SURCHG** → 新增兩筆：**AA** / 啟用 / **15.00**、**LH** / 啟用 / **10.00**
- 驗證重用標準型別的三个免費好處：
 1. **CARRID** 欄位按 **F4**：選單直接出現（航空公司清單）——因為 **S_CARR_ID** 早就掛好了標準 Search Help，我們什麼都沒建
 2. 輸入一個 **SCARR** 沒有的代碼（如 **ZZ**）：畫面直接擋下來——外鍵在起作用，Check Table 是標準表一樣有效
 3. **ACTIVE** 欄位有正確的欄位標題（不是通用符號 +），按 **F4** 出現「是 / 否」選單——重用 **XFELD** 的固定值清單換來的，一個 Domain 都沒自己建

對照講義 21：**ZTR21_STUD-KLASSE** 因為是全新自建概念，Search Help 要整個手工建；這裡因為重用了標準 Data Element，Search Help 完全不用建——這就是 Global Type 省下的工。

6.5 程式讀取：Global Type 宣告 + 加成營收試算

本講排在 FORM（講義 8）與 JOIN（講義 11）之前，所以程式全部寫在 **START-OF-SELECTION**，用前面學過的「三張表各自讀進內表、再用 **READ TABLE** 對照」完成：

```
DATA: gt_surchg TYPE ztr25_tt_surchg,      " DDIC Table Type ( 見 6.6 )
      gs_surchg TYPE ztr25_surchg,        " 整列：Global Type
      gt_flight TYPE STANDARD TABLE OF sflight,
      gs_flight TYPE sflight,
      gt_scarr TYPE STANDARD TABLE OF scarr,
      gs_scarr TYPE scarr,
      gt_rev TYPE STANDARD TABLE OF ty_rev,      " ty_rev：航班欄位 + active /
surcharge_pct / revenue / revenue_adj
      gs_rev TYPE ty_rev.

SELECT * FROM sflight INTO TABLE gt_flight
  WHERE seatsocc > 0
  ORDER BY carrid connid fldate.
SELECT * FROM scarr INTO TABLE gt_scarr.
SELECT * FROM ztr25_surchg INTO TABLE gt_surchg.

LOOP AT gt_flight INTO gs_flight.
```

```

CLEAR gs_rev.
gs_rev-carrid   = gs_flight-carrid.
gs_rev-connid   = gs_flight-connid.
gs_rev-fldate   = gs_flight-fldate.
gs_rev-seatsocc = gs_flight-seatsocc.
gs_rev-price    = gs_flight-price.

READ TABLE gt_scarr INTO gs_scarr WITH KEY carrid = gs_flight-carrid.
IF sy-subrc = 0.
    gs_rev-carrname = gs_scarr-carrname.
ENDIF.

READ TABLE gt_surchg INTO gs_surchg WITH KEY carrid = gs_flight-carrid.
IF sy-subrc = 0.
    gs_rev-active      = gs_surchg-active.
    gs_rev-surcharge_pct = gs_surchg-surcharge_pct.
ENDIF.                                " 找不到 = 沒設定，維持初始值

gs_rev-revenue = gs_rev-price * gs_rev-seatsocc.
IF gs_rev-active = 'X'.
    gs_rev-revenue_adj = gs_rev-revenue * ( 1 + gs_rev-surcharge_pct / 100 ).
ELSE.
    gs_rev-revenue_adj = gs_rev-revenue.
ENDIF.
APPEND gs_rev TO gt_rev.
ENDLOOP.

```

關鍵在第二個 **READ TABLE**：沒被財務設定過的航空公司找不到 (**sy-subrc** <> 0)，**active** / **surcharge_pct** 維持初始值，**revenue_adj** 自然等於原始營收，這筆航班照樣出現在報表上。

講義 11 會學到 JOIN：這種「對照另一張表、找不到也要保留」的讀法，可以用一句 **LEFT OUTER JOIN** 完成。到時候回頭對照這段，就知道 JOIN 省了什麼。

6.6 把「表格型別」也升級成 Global Type：建立 DDIC Table Type

講義 4 教過 **TYPES tt_student TYPE STANDARD TABLE OF ty_student**——但那是 **Local Type**，只有寫在那支程式裡看得到。如果**兩支不同程式都要用同一種表格**，各自宣告一份 Local Type 只是把同一件事寫兩遍——改一個欄位，兩邊都要記得改，正是 Global Type 一開始要解決的問題（第 2 節），只是這次問題發生在「表格」這個層級，不是單一欄位。

解法是 SE11 建一個**全域的 Table Type**（DDIC 物件型別 **TTYP**），跟 Domain / Data Element 一樣只寫一次、全系統共用：

1. SE11 → Data Type → 輸入 **ZTR25_TT_SURCHG** → Create → 選 **Table Type**
2. **Line Type**（每一列的長相）：直接填 **ZTR25_SURCHG**——表格本身也可以拿來當 **Line Type**，不用重新定義欄位。這是另一層 Global Type 的好處：以後 **ZTR25_SURCHG** 加欄位，**ZTR25_TT_SURCHG** 自動跟著多那個欄位，兩邊零維護成本

3. **Access Mode**：選 **Standard Table** (對照講義 4：STANDARD / SORTED / HASHED 三選一)，Key 用預設 (Default Key) 即可
4. 存檔、套件 **\$TMP**、啟用

用起來跟本地表格型別語法一模一樣，只是型別名稱換成 DDIC 物件：

```
DATA gt_surchg TYPE ztr25_tt_surchg.      " 引用全域 Table Type，跟 TYPE STANDARD
TABLE OF ztr25_surchg 效果相同

SELECT * FROM ztr25_surchg INTO TABLE gt_surchg.
```

任何程式都能宣告 **TYPE ztr25_tt_surchg**，拿到的是同一份定義——這就是「表格級別」的 Global Type。

它真正發揮作用的地方，是當「參數型別」：接下來講義 8 的 FORM 可以用它當 **CHANGING** 參數；講義 15 的 Function Module 更是一定要用 DDIC 型別定義介面參數 (FM 介面不能程式裡自己宣告的 **TYPES**)，用 **CHANGING** 搭配 Table Type 傳一整張表，正是取代舊式 **TABLES** 參數的寫法。所以本講先學會建 Table Type，後面兩講直接用得上。

判斷原則跟前面幾種模式一致：只有本程式用就地 **TYPES** 宣告 (講義 4) 就好，不用每個表都跑一次 SE11；多支程式 / FM / 方法要共用同一種表格參數，才值得升級成 DDIC Table Type。

7. 常見錯誤與陷阱

症狀	原因
SAP 升級後某欄位資料被截斷	程式宣告用了寫死的長度，沒有引用 Global Type (2.1 的真實案例)
自建 Data Element 卻發現標準早就有一個一模一樣的	建之前沒有先搜尋標準——語意重複的型別會讓維護更亂
外鍵 Check Table 選了自建 Z 表，其實應該指標準表	沒想清楚「合法值清單本來就已經存在」 (如本例的 SCARR)
F4 選單自己手動建了一個 Search Help，其實不用	沒發現重用的 Data Element 早就掛好標準 Search Help
沒設定的航空公司整筆消失	找不到設定時直接 CONTINUE 跳過了—— READ TABLE 找不到要保留、維持初始值 (學完講義 11 後，對應的是誤把 LEFT OUTER JOIN 寫成 INNER JOIN)
Table Maintenance Generator 報 Function Group 不存在	要先在 SE80 建好 Function Group (見 6.4)
SM30 資料改了以為要走傳輸請求	表 結構 才需要 TR；表 資料 維護不需要 (除非 Delivery Class 特別設定)
SM30 欄位標題顯示通用符號 +、也沒有 F4 選單	欄位直接用內建型別 (如 CHAR 1)，沒有掛任何 Data Element——通用旗標欄位改用「重用標準 Domain (如 XFELD) + 自建 Data Element 補標籤」 (見 6.2)

症狀	原因
每支程式都各自宣告一份幾乎一樣的 TYPES tt_xxx TYPE STANDARD TABLE OF ...	這種表格型別其實多支程式在共用，該升級成 DDIC Table Type (見 6.6)，而不是各自維護一份 Local Type

8. 課堂練習

完成 [ex25](#)：建自訂 Domain/DE (加成百分比) + 一張重用標準 Data Element 當 Key 又當外鍵的表、Check Table 指向標準 [SCARR](#)、SM30 維護畫面、一個引用該表當 Line Type 的 DDIC Table Type，再寫程式驗證「重用型別免費拿到 F4 與外鍵檢查」，並用 Global Type 宣告 (含表格級的 Table Type) 完成一份加成營收報表。